Enterprise Learning Platform

with Skill and Career Guidance System

Milestone 1 — Project Documentation

Foundation: Authentication & Landing Experience

Team C

Prepared for team review

1. Overview

Milestone 1 establishes the foundation of the Enterprise Learning Platform with Skill and Career Guidance System: a working Google OAuth2 + JWT login flow, a MySQL database with a users table, and a fully designed themed landing page. No dashboard, no courses, and no learning content exist yet — this milestone is, by design, an authentication skeleton with a UI shell around it, built to prove the core login and security pipeline works end to end before any learning features are layered on top.

2. Technology Stack

Layer	Technology	Notes
Frontend	React (Vite)	JavaScript, not TypeScript
Backend	Java 17 + Spring Boot 3.5.16	Maven build (mvnw)
Database	MySQL	Local MySQL during development; cloud MySQL migration planned
Authentication	Google OAuth2 (OIDC) + JWT	Login via Google; JWT used for API session/auth
Frontend routing	react-router-dom	Added to handle the /oauth-success callback page

Why this stack: the backend language was locked to Java early on, so Spring Boot was chosen because it has first-class, well-documented support for exactly this combination — Google OAuth2 login, JWT authentication, and MySQL via Spring Data JPA/Hibernate. React (Vite) was chosen for the frontend because it pairs cleanly with a Spring Boot REST API and has abundant reference material for "Google login + JWT" flows.

3. High-Level Architecture

There are three moving parts:

- Frontend (client/) — the React app the user sees: landing page, "Continue with Google" button, and a small page that catches the login result.
- Backend (server/) — a Spring Boot app that talks to Google, issues JWT tokens, and exposes protected API endpoints.
- Database (MySQL) — stores a single users table right now (id, email, google_id, name, profile_picture), auto-created by Hibernate from the User entity.

3.1 How Login Works, Step by Step

- User clicks "Continue with Google" on the React landing page.
- Browser is redirected to Google's real sign-in page (standard OAuth2/OIDC, not a custom login form).

- User logs in and consents; Google redirects back to the backend at /login/oauth2/code/google with an authorization code.
- Spring Security exchanges that code with Google for the user's profile (email, name, Google ID, picture).
- CustomOAuth2UserService checks the users table: if the email already exists it reuses that row, otherwise it creates a new user row.
- OAuth2LoginSuccessHandler generates a JWT for that user and redirects the browser to the frontend at /oauth-success?token=<jwt>.
- The React OAuthSuccess.jsx page reads the token from the URL, stores it via AuthContext, and redirects the user back to the homepage as "logged in".
- From then on, the frontend sends that JWT in the Authorization: Bearer <token> header on every protected request; JwtAuthenticationFilter validates it on every request.

This entire flow has been tested end-to-end: Google login → user saved in MySQL → JWT issued → protected endpoint accepts the token and rejects requests without one.

4. Backend Files & Responsibilities

File	Purpose
model/User.java	JPA entity defining the users table columns: id, email, google_id, name, profile_picture.
repository/UserRepository.java	Spring Data JPA interface; adds findByEmail and findByGoogleId finders.
security/JwtUtil.java	Creates a signed JWT for a given email and validates/parses a token later.
security/CustomOAuth2UserService.java	Runs after Google confirms identity; finds or creates the matching user row.
security/OAuth2LoginSuccessHandler.java	Generates a JWT on successful login and redirects to the frontend with the token.
security/JwtAuthenticationFilter.java	Validates the Authorization header JWT on every incoming protected request.
config/SecurityConfig.java	Central security wiring: public vs. protected routes, CORS, OAuth2 login configuration.
controller/TestController.java	Protected GET /api/profile endpoint used to prove the whole chain works.

5. Frontend Files & Responsibilities

File	Purpose
context/AuthContext.jsx	Global login state — token, login(), logout(), isAuthenticated — mirrored into sessionStorage.

File	Purpose
components/Navbar.jsx	Top navigation; shows Sign in / Sign out based on auth state.
components/Hero.jsx	Landing hero section.
components/Routes.jsx	Sample course previews shown on the landing page.
components/Instruments.jsx	Feature highlights section on the landing page.
components/TrailLog.jsx	Placeholder analytics/progress section on the landing page.
components/Connect.jsx	The real “Continue with Google” button, wired to the backend.
components/OAuthSuccess.jsx	Catches ?token=... after Google login and stores it via AuthContext.

6. Database

Database name: the platform's primary MySQL database, running locally during development, with a cloud MySQL migration planned for a later milestone. Hibernate (spring.jpa.hibernate.ddl-auto=update) automatically creates and updates the users table from User.java — no manual SQL is required at this stage.

7. Current Status

Area	Status
Google OAuth2 login flow (redirect to Google → consent → redirect back)	Complete & verified
New users correctly created in MySQL; returning users correctly recognized by email	Complete & verified
JWT generated on login and passed to frontend via redirect URL	Complete & verified
Protected backend endpoint (/api/profile) enforces valid-token access	Complete & verified
Frontend catches JWT, stores it, and reflects logged-in/out state in the UI	Complete & verified
Themed landing page built and rendering correctly	Complete & verified
Dashboard, course catalog, learning content	Not started — planned for Milestone 2

8. Recommended Next Steps

- Build a real dashboard and course catalog backed by actual database tables (Milestone 2).

- Migrate the database from local MySQL to a managed cloud MySQL instance, including SSL handling.
- Return a clean 401 JSON response for unauthenticated API calls instead of redirecting to the Google login page, once more API-driven pages exist beyond login.
- Rotate the Google OAuth client secret, JWT secret, and MySQL password before any shared/public use — treat anything shared during development or debugging as compromised.